

VISVESVARAYA TECHNOLOGICAL UNIVERSITY**“Jnana Sangama”, Machhe, Belagavi, Karnataka-590018**

Lab Experiment Record

Project Management with Git [BCSL58C]*Submitted in partial fulfillment towards AEC of 3rd semester of***Bachelor of Engineering
in
Computer Science and Engineering
(Artificial Intelligence & Machine Learning)**

Submitted by

**SHRUTHI REFDY M
4GW24CI028****DEPARTMENT OF CSE (Artificial Intelligence & Machine Learning)****GSSS INSTITUTE OF ENGINEERING & TECHNOLOGY FOR WOMEN****(Affiliated to VTU, Belagavi, Approved by AICTE, New Delhi & Govt. of Karnataka)****K.R.S ROAD, METAGALLI, MYSURU-570016, KARNATAKA****(Accredited by NAAC)****2025-2026**

Project Management with Git		Semester	3
Course Code	BCS358C	CIE Marks	50
Teaching Hours/Week (L:T:P: S)	0: 0 : 2: 0	SEE Marks	50
Credits	01	Exam Marks	100
Examination type (SEE)	Practical		
Course objectives:			
• .To familiar with basic command of Git • To create and manage branches • To understand how to collaborate and work with Remote Repositories • To familiar with virion controlling commands			
SLNO	Experiments		
1	Setting Up and Basic Commands Initialize a new Git repository in a directory. Create a new file and add it to the staging area and commit the changes with an appropriate commit message.		
2	Creating and Managing Branches Create a new branch named "feature-branch." Switch to the "master" branch. Merge the "feature-branch" into "master."		
3	Creating and Managing Branches Write the commands to stash your changes, switch branches, and then apply the stashed changes.		
4	Collaboration and Remote Repositories Clone a remote Git repository to your local machine.		
5	Collaboration and Remote Repositories Fetch the latest changes from a remote repository and rebase your local branch onto the updated remote branch.		
6	Collaboration and Remote Repositories Write the command to merge "feature-branch" into "master" while providing a custom commit message for the merge.		
7	Git Tags and Releases Write the command to create a lightweight Git tag named "v1.0" for a commit in your local repository.		
8	Advanced Git Operations		

	Write the command to cherry-pick a range of commits from "source-branch" to the current branch.
9	Analysing and Changing Git History Given a commit ID, how would you use Git to view the details of that specific commit, including the author, date, and commit message?
10	Analysing and Changing Git History Write the command to list all commits made by the author "JohnDoe" between "2023-01-01" and "2023-12-31."
11	Analysing and Changing Git History Write the command to display the last five commits in the repository's history.
12	Analysing and Changing Git History Write the command to undo the changes introduced by the commit with the ID "abc123".
Course outcomes (Course Skill Set): At the end of the course the student will be able to: • Use the basics commands related to git repository • Create and manage the branches • Apply commands related to Collaboration and Remote Repositories • Use the commands related to Git Tags, Releases and advanced git operations • Analyse and change the git history	

Course Contents :

- 1. Git Basics.*
- 2. Git Installation*
- 3. Git Basic Commands*
- 4. Experiments*
 - 1. Setting Up and Basic Commands Initialize a new Git repository in a directory. Create a new file and add it to the staging area and commit the changes with an appropriate commit message.*
 - 2. Creating and Managing Branches Create a new branch named "feature-branch." Switch to the "master" branch. Merge the "feature-branch" into "master."*
 - 3. Creating and Managing Branches Write the commands to stash your changes, switch branches, and then apply the stashed changes.*
 - 4. Collaboration and Remote Repositories Clone a remote Git repository to your local machine.*

5. Collaboration and Remote Repositories Fetch the latest changes from a remote repository and rebase your local branch onto the updated remote branch.

6. Collaboration and Remote Repositories Write the command to merge "feature-branch" into "master" while providing a custom commit message for the merge.

7. Git Tags and Releases Write the command to create a lightweight Git tag named "v1.0" for a commit in your local repository.

8. Advanced Git Operations Write the command to cherry-pick a range of commits from "source-branch" to the current.

9. Analysing and Changing Git History Given a commit ID, how would you use Git

Program 1: Setting Up and Basic Commands

Commands Used:

1. Initialize a new Git repository:
 - Git clone
 - `cd git_lab`
 - `git init`
2. Create a new file:
 - `touch lab1.txt`
3. Add the file to the staging area:
 - `git add lab1.txt`
4. Commit the changes with a message:
 - `git commit -m "Initial commit - adding a new file"`
 - `git push origin main`
use the 'status' command so we can see what git is tracking.
`git status`

Add the new file using the 'add' command.

Git add .

The '.' Adds all the files in your folder in this case only the README.md file

git clone: <https://github.com/shruthireddy8286-coder/4GW24CI028.git>

```

GSSS@DESKTOP-G36BAK2 MINGW64 ~/Documents/git report
$ git init
Initialized empty Git repository in C:/Users/GSSS/Documents/git report/.git/

GSSS@DESKTOP-G36BAK2 MINGW64 ~/Documents/git report (master)
$ ls -a
./ ../ .git/

GSSS@DESKTOP-G36BAK2 MINGW64 ~/Documents/git report (master)
$ touch README.md

GSSS@DESKTOP-G36BAK2 MINGW64 ~/Documents/git report (master)
$ echo "First file"> README.md

GSSS@DESKTOP-G36BAK2 MINGW64 ~/Documents/git report (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md

nothing added to commit but untracked files present (use "git add" to track)

GSSS@DESKTOP-G36BAK2 MINGW64 ~/Documents/git report (master)
$ git add README.md
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it

GSSS@DESKTOP-G36BAK2 MINGW64 ~/Documents/git report (master)
$ git add .

```

Commit the changes using 'commit' command.

```
git commit -m "Initial Commit"
```

```

GSSS@DESKTOP-G36BAK2 MINGW64 ~/Documents/git report (master)
$ git commit -m "Initial Commit"
[master (root-commit) 7f35935] Initial Commit
1 file changed, 1 insertion(+)
create mode 100644 README.md

```

You can confirm the commit using the 'log --oneline' message and view the content of the file using 'cat' command.

```
git log --oneline
```

```
cat README.md
```

```

GSSS@DESKTOP-G36BAK2 MINGW64 ~/Documents/git report (master)
$ git log --oneline
b0690fc (HEAD -> master) Second Commit
7f35935 Initial Commit

GSSS@DESKTOP-G36BAK2 MINGW64 ~/Documents/git report (master)
$ cat README.md
# First Line

```

Program 2: Creating and Managing Branches Create a new branch named "feature-branch." Switch to the "master" branch. Merge the "feature-branch" into "master."

The command is: git branch feature-branch

```
shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ touch README.md

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git add.
git: 'add.' is not a git command. See 'git --help'.

The most similar command is
    add

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git commit -m "readme file"
[master 5740103] readme file
1 file changed, 1 insertion(+)
create mode 160000 gsss/aiml_repo

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git status
On branch master
Changes not staged for commit:
```

‘checkout’ command.

The command is: git checkout main

To go back to the feature-branch we have to use: git checkout feature-branch

we use the ‘merge’ command to merge both the branches.

git m

erge feature-branch

```
shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git add README.md

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ ls
Git_Lab/ README.md

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git branch feature-branch

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git checkout main
error: pathspec 'main' did not match any file(s) known to git

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git merge feature-branch
Already up to date.

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$
```

Program 3: Creating and Managing Branches Write the commands to stash your changes, switch branches, and then apply the stashed changes.

git stash push -m "WIP: my changes"

(WIP- work in progress)

```
shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git init
Initialized empty Git repository in C:/Users/shrut/OneDrive/Desktop/4GW24CI028/.git/

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git config user.email
shruthireddy8286@gmail.com

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git config user.name
shruthireddy8286-coder

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ |
```

```
shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ touch file.txt

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md
        file.txt

nothing added to commit but untracked files present (use "git add" to track)

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git status push -m "WIP:..my changes"
error: unknown switch `m'
```

git checkout -b target-branch

restore the changes we have made in our previous branch to this branch,

git stash pop

```
shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git check -b target-branch
git: 'check' is not a git command. See 'git --help'.

The most similar command is
    checkout

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git stash pop
No stash entries found.

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ |
```

```
shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git check -b target-branch
git: 'check' is not a git command. See 'git --help'.

The most similar command is
  checkout

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git stash pop
No stash entries found.

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git status
On branch master

No commits yet
```

Program 4: Collaboration and Remote Repositories Clone a remote Git repository to your local machine.

git clone: <https://github.com/shruthireddy8286-coder/4GW24CI028.git>

```
shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git clone https://github.com/ShruthiReddy/4GW24CI028.git
Cloning into '4GW24CI028'...
remote: Repository not found.
fatal: repository 'https://github.com/ShruthiReddy/4GW24CI028.git/' not found
```

can change our directory to the name of our repository.

cd 4GW24CI003

```
shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ cd 4GW24CI028
bash: cd: 4GW24CI028: No such file or directory
```

Program 5: Collaboration and Remote Repositories Fetch the latest changes from a remote repository and rebase your local branch onto the updated remote branch.

```
shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git status
On branch master

no commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md
        file.txt

nothing added to commit but untracked files present (use "git add" to track)

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git commit -m "commit these changes"
On branch master

initial commit

Untracked files:
```

git fetch origin

switch to the target-branch. Then rebase local branch onto remote.

git checkout target-branch

git rebase origin/main

```
shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git checkout target-branch
error: pathspec 'target-branch' did not match any file(s) known to git

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ rebase origin/main
origin/main: skipped because nonexistent.
```

```
shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ ls
Git_Lab/  README.md  file.txt
```

Program 6: Collaboration and Remote Repositories Write the command to merge "feature-branch" into "master" while providing a custom commit message for the merge

git merge -m "Merge feature-branch: add login feature" feature-branch

```
shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git check -b master
git: 'check' is not a git command. See 'git --help'.

The most similar command is
    checkout

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git check -b feature-branch
git: 'check' is not a git command. See 'git --help'.

The most similar command is
    checkout

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git checkout master
error: pathspec 'master' did not match any file(s) known to git
```

```
shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ ls
Git_Lab/  README.md  file.txt
```

Program 7: Git Tags and Releases Write the command to create a lightweight Git tag named "v1.0" for a commit in your local repository.

```
shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git tag v1.0
fatal: Failed to resolve 'HEAD' as a valid ref.

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git log --oneline
bash: git: command not found

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git log --oneline
fatal: your current branch 'master' does not have any commits yet
```

Using 'log' command we can see the commit with the tag we have used.

git log --oneline

Program 8: Advanced Git Operations: Write the command to cherry-pick a range of commits from "source-branch" to the current branch

```
git log source-branch --oneline -10
```

Using this command, we can see the commits with their id's. After this we use the 'cherry-pick' command. We give the range from the oldest id to the newest id.

```
git cherry-pick eea29d3^..b8112df
```

```
shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git log source-branch --oneline -10
fatal: ambiguous argument 'source-branch': unknown revision or path not in the working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git cherry-pick eea29d3^..b819c04
fatal: bad revision 'eea29d3^..b819c04'

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git cherry-pick --abort
error: no cherry-pick or revert in progress
fatal: cherry-pick failed

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ |
```

Program 9: Analysing and Changing Git History Given a commit ID, how would you use Git to view the details of that specific commit, including the author, date, and commit message?

To view the history and all other details we use only one command that is the 'show' command

```
shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ git show b819c04
fatal: ambiguous argument 'b819c04': unknown revision or path not in the working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'

shrut@Reddy MINGW64 ~/OneDrive/Desktop/4GW24CI028 (master)
$ |
```

Program 10: Analysing and Changing Git History Write the command to list all commits made by the author "JohnDoe" between "2023-01-01" and "2023-12-31."

To list all commits made by the author "JohnDoe" between "2023-01-01" and "2023-12-31" in Git, you can use the git log command with the --author and --since and --until options. Here's the command:

```
git log --author="JohnDoe" --since="2023-01-01" --until="2023-12-31"
```

```
shrut@Reddy MINGW64 ~ (main)
$ git log --author="JohnDoe" --oneline

shrut@Reddy MINGW64 ~ (main)
$ git log --author="JohnDoe" --pretty=format:"%h %an %ad %s"

shrut@Reddy MINGW64 ~ (main)
$ git log -p
commit 792e1b12a2b80dc1783d3cab32ce02775eee0c9 (HEAD -> main)
Author: sweety <shruthireddy8286@gmail.com>
Date:   Mon Sep 8 12:46:41 2025 +0530

    first commit

diff --git a/README.md b/README.md
new file mode 100644
index 0000000..d5d557a
--- /dev/null
+++ b/README.md
@@ -0,0 +1 @@
+# nikki
```

```
shrut@Reddy MINGW64 ~ (main)
$ git rebase -i HEAD~3
fatal: invalid upstream 'HEAD~3'

shrut@Reddy MINGW64 ~ (main)
$ git log --oneline --graph --all
* 792e1b1 (HEAD -> main) first commit

shrut@Reddy MINGW64 ~ (main)
$ git log --reverse
commit 792e1b12a2b80dc1783d3cab32ce02775eee0c9 (HEAD -> main)
Author: sweety <shruthireddy8286@gmail.com>
Date: Mon Sep 8 12:46:41 2025 +0530

    first commit
```

This command will display a list of commits made by the author "JohnDoe" that fall within the specified date range, from January 1, 2023, to December 31, 2023. Make sure to adjust the author's

Program 11: Analysing and Changing Git History Write the command to display the last five commits in the repository's history.

`git log -n 5`

This command will show the last five commits in the repository's history. You can adjust the number after -n to display a different number of commits if needed.

```

Shruthireddy@Ace_Anju MINGW64 /e/Documents/git programs (master)
git log -5 --oneline
commit 8aca19605910800ec303bb546a1687b1296e2a5b (HEAD -> master)
  Author: Shruthireddy <shruthireddy8286@gmail.com> o:78 commit
    Mon Jan 5 18:08:24 2026 +0530

Sample commit 4

commit c0a167adc3fda734bf9d88c93335c53924eeac9ff8
  Author: Shruthireddy <shruthireddy8286@gmail.com>
    Mon Jan 5 18:08:24 2026 +0530

Sample commit 5

commit 8aca19605910800ec03bb546a16873b1296e2a5b (HEAD -> master)
  Author: Shruthireddy <shruthireddy8286@gmail.com> 0:78 commit
    Mon Jan 5 18:08:24 2026 +0530

Sample commit 3

commit c095a5294bf712039ba38479df44896a7aef58d9
  Author: Shruthireddy <shruthireddy8286@gmail.com>
    Mon Jan 5 18:08:24 2026 +0530

Sample commit 3

commit 8aca19605910800ec303bb546a1687b1296e2a5b
  Author: Shruthireddy <shruthireddy8286@gmail.com>
    Mon Jan 5 18:08:24 2026 +0530

Sample commit 5

commit c095a5294bf712039ba38479bf4489e7aef58d9
  Author: Shruthireddy <shruthireddy8286@gmail.com>
    Mon Jan 5 18:08:24 2026 +0530

shruthireddy88899156afb30dcbff1b97898 (tag: v1.0, target-branch)
committing first file: shruthireddy8286@gmail.com

```

Program 12: Analyzing and Changing Git History Write the command to undo the changes introduced by the commit with the ID "abc123".

To undo the changes introduced by a specific commit with the ID "abc123" in Git, you can use the git revert command. The git revert command creates a new commit that undoes the changes made by the specified commit, effectively "reverting" the commit. Here's the command:

```
git revert abc123
```

```
shrut@Reddy MINGW64 ~ (main)
$ git revert abc123
fatal: bad revision 'abc123'

shrut@Reddy MINGW64 ~ (main)
$ git reset --soft abc123^
fatal: ambiguous argument 'abc123^': unknown revision or path not in the working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'

shrut@Reddy MINGW64 ~ (main)
$ git reset --mixed abc123^
fatal: ambiguous argument 'abc123^': unknown revision or path not in the working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'

shrut@Reddy MINGW64 ~ (main)
$ git reset abc123^
fatal: ambiguous argument 'abc123^': unknown revision or path not in the working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'

shrut@Reddy MINGW64 ~ (main)
$ |
```

Replace "abc123" with the actual commit ID that you want to revert. After running this command, Git will create a new commit that negates the changes introduced by the specified commit. This is a safe way to undo changes in Git because it preserves the commit history and creates a new commit to record the reversal of the changes.

```
shrut@Reddy MINGW64 ~ (main)
$ git checkout abc123^ -- file.txt
fatal: invalid reference: abc123^

shrut@Reddy MINGW64 ~ (main)
$ git rebase -i abc123^
fatal: invalid upstream 'abc123^'

shrut@Reddy MINGW64 ~ (main)
$ pick abc123 commit-message
bash: pick: command not found

shrut@Reddy MINGW64 ~ (main)
$ drop abc123 commit-message
bash: drop: command not found

shrut@Reddy MINGW64 ~ (main)
$ git log --oneline
792e1b1 (HEAD -> main) first commit

shrut@Reddy MINGW64 ~ (main)
$ git revert --abort
```